

Chapter3: Problem solving paradigms - 1

United International College

Outline

- Problem solving paradigms
- Complete search
 - Iterating
 - Backtracking (Recursion)
- Divide and conquer
 - Binary search
 - Binary exponentiation
 - Fibonacci words

Problem solving paradigms

- What is a problem solving paradigm?
- A method to construct a solution to a specific type of problem
- Today and in later lectures we will study common problem solving paradigms

Complete search

We have a finite set of objects

We want to find an element in that set which satisfies some constraints
– or find all elements in that set which satisfy some constraints

Simple! Just go through all elements in the set, and for each of them check if they satisfy the constraints

Of course it's not going to be very efficient...

But remember, we always want the simplest solution that runs in time

Complete search should be the first problem solving paradigm you think about when you're trying to solve a problem

Example problem: UVa10041 Vito's family

The world-known gangster Vito Deadstone is moving to New York. He has a very big family there, all of them living in Lamafia Avenue. Since he will visit all his relatives very often, he is trying to find a house close to them.

Vito wants to minimize the total distance to all of them and has blackmailed you to write a program that solves his problem.

Input

The input consists of several test cases. The first line contains the number of test cases.

For each test case you will be given the integer number of relatives r ($0 < r < 500$) and the street numbers (also integers) $s_1, s_2, \dots, s_i, \dots, s_r$ where they live ($0 < s_i < 30000$). Note that several relatives could live in the same street number.

Output

For each test case your program must write the minimal sum of distances from the optimal Vito's house to each one of his relatives. The distance between two street numbers s_i and s_j is $d_{ij} = |s_i - s_j|$.

Sample Input

```
2
2 2 4
3 2 4 6
```

Sample Output

```
2
4
```

Complete search

What if the search space is more complex?

- All permutations of n items
- All subsets of n items
- All ways to put n queens on an $n \times n$ chessboard without any queen attacking any other queen

How are we supposed to iterate through the search space?

Let's take a better look at these examples

Iterating through permutations

- ▶ Already implemented in many standard libraries:

- `next_permutation` in C++
- `itertools.permutations` in Python

```
int n = 5;
vector<int> perm(n);
for (int i = 0; i < n; i++) perm[i] = i + 1;

do {
    for (int i = 0; i < n; i++) {
        printf("%d ", perm[i]);
    }
    printf("\n");
} while (next_permutation(perm.begin(), perm.end()));
```

- ▶ Even simpler in Python...

- ▶ Remember that there are $n!$ permutations of length n , so usually you can only go through all permutations if $n \leq 11$
 - Otherwise you need to find a more clever approach than complete search

Iterating through subsets

- ▶ Remember the bit representation of subsets?
- ▶ Each integer from 0 to $2^n - 1$ represents a different subset of the set $\{1, 2, \dots, n\}$
- ▶ Just iterate through the integers

```
int n = 5;
for (int subset = 0; subset < (1 << n); subset++) {
    for (int i = 0; i < n; i++) {
        if ((subset & (1 << i)) != 0) {
            printf("%d ", i+1);
        }
    }
    printf("\n");
}
```

- ▶ Similar in Python
- ▶ Remember that there are 2^n permutations of length n , so usually you can only go through all permutations if $n \leq 25$
 - Otherwise you need to find a more clever approach than complete search

Iterating through permutations

UVa11742–SocialConstraints

- There are $0 < n \leq 8$ moviegoers
- They will sit in the front row with n consecutive open seats
- There are $0 \leq m \leq 20$ seating constraints among them,
i.e. a and b must be at most (or at least) c seats apart
- How many possible seating arrangements are there?

Iterating through permutations

UVa11742–SocialConstraints

- There are $0 < n \leq 8$ moviegoers
 - They will sit in the front row with n consecutive open seats
 - There are $0 \leq m \leq 20$ seating constraints among them,
i.e. a and b must be at most (or at least) c seats apart
 - How many possible seating arrangements are there?
- Iterative Complete Search Solution (Permutations):
 - Set counter = 0 and then try all possible $n!$ permutations
 - Increase counter if a permutation satisfies all m constraints
 - Output the final value of counter

Iterating through subsets

- UVa 12346 – Water Gate Management
 - A dam has $1 \leq n \leq 20$ water gates to let out water when necessary, each water gate has flow rate and damage cost
 - Your task is to manage the opening of the water gates in order to get rid of at least the specified total flow rate condition that the total damage cost is minimized!

Iterating through subsets

- UVa 12346 – Water Gate Management
 - A dam has $1 \leq n \leq 20$ water gates to let out water when necessary, each water gate has flow rate and damage cost
 - Your task is to manage the opening of the water gates in order to get rid of at least the specified total flow rate condition that the total damage cost is minimized!
- Iterative Complete Search Solution (Subsets):
 - Try all possible 2^n subsets of water gates to be opened
 - For each subset, check if it has sufficient flow rate
 - If it is, check if the total damage cost of this subset is smaller than the overall minimum damage cost so far
 - If it is, update the overall minimum damage cost so far
 - Output the minimum damage cost

Backtracking

- We've seen two ways to go through a complex search space, but both of the solutions were rather specific
- Would be nice to have a more general “framework”
- Backtracking!

Backtracking

Define states

- We have one initial “empty” state
- Some states are partial
- Some states are complete

Define transitions from a state to possible next states

Basic idea:

1. Start with the empty state
2. Use recursion to traverse all states by going through the transitions
3. If the current state is invalid, then stop exploring this branch
4. Process all complete states (these are the states we’re looking for)

Backtracking

General solution form:

```
state S;

void generate() {
    if (!is_valid(S))
        return;

    if (is_complete(S))
        print(S);

    foreach (possible next move P) {
        apply move P;
        generate();
        undo move P;
    }
}

S = empty state;
generate();
```

Generating all subsets

Also simple to do with backtracking:

```
const int n = 5;
bool pick[n];

void generate(int at) {
    if (at == n) {
        for (int i = 0; i < n; i++) {
            if (pick[i]) {
                printf("%d ", i+1);
            }
        }
        printf("\n");
    } else {
        // either pick element no. at
        pick[at] = true;
        generate(at + 1);

        // or don't pick element no. at
        pick[at] = false;
        generate(at + 1);
    }
}

generate(0);
```

Generating all permutations

Also simple to do with backtracking:

```
const int n = 5;
int perm[n];
bool used[n];

void generate(int at) {
    if (at == n) {
        for (int i = 0; i < n; i++) {
            printf("%d ", perm[i]);
        }
        printf("\n");
    } else {
        // decide what the at-th element should be
        for (int i = 0; i < n; i++) {
            if (!used[i]) {
                used[i] = true;
                perm[at] = i;

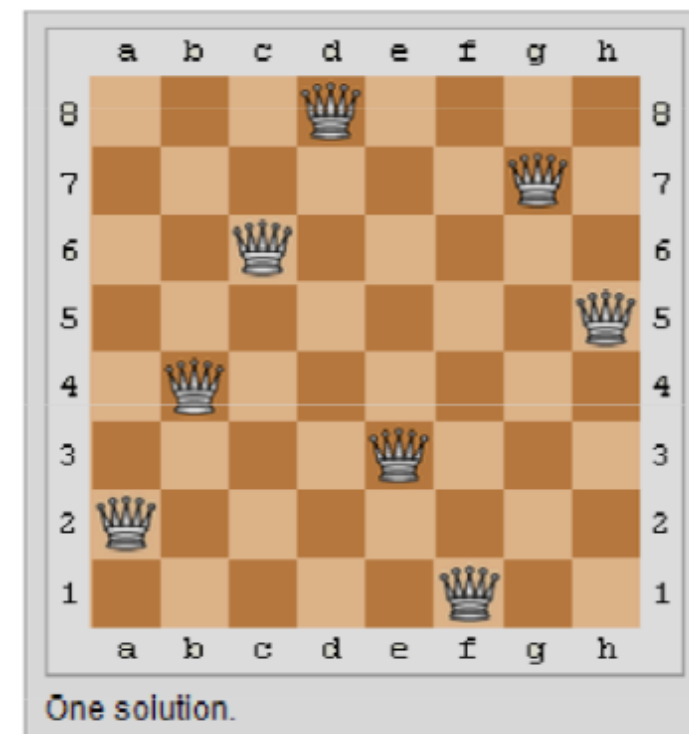
                generate(at + 1);

                // remember to undo the move:
                used[i] = false;
            }
        }
    }
}

memset(used, 0, n);
generate(0);
```

8 queens

- UVa750–8 Queens Chess Problem
 - Put 8 queens in 8x8 Chessboard
 - No queen can attack other queens
- Naïve ways (Time Limit Exceeded)
 - Choose 8 out of 64 cells...
 - $C_{64}^8 = 4$ Billion possibilities...
 - Insight 1: Put one queen in each column...
 - $8^8 = 17$ Million possibilities... :O



8 queens

- Better way, recursive backtracking
- Insight 2: **all-different constraint** for the rows too
 - We put one queen in each column **AND each row**
 - Finding a valid permutation out of $8!$ possible permutations...
 - Search space goes down from $8^8 = 17\text{M}$ to $8! = 40\text{K}$!
- Insight 3: **main diagonal and secondary diagonal** check
 - Another way to prune the search space
 - Queen A (i, j) attacks Queen B (k, l) iff
$$\text{abs}(i - k) == \text{abs}(j - l)$$

Scrutinize the sample code of recursive backtracking!

n queens

Go through the cells in increasing order

Either put a queen on that cell or not (transition)

Don't put down a queen if she's able to attack another queen already on the table

```
const int n = 8;
bool has_queen[n][n];
int queens_left = n;

// generate function

memset(has_queen, 0, sizeof(has_queen));
generate(0, 0);
```

n queens

```
void generate(int x, int y) {
    if (y == n) {
        generate(x+1, 0);
    } else if (x == n) {
        if (queens_left == 0) {
            for (int i = 0; i < n; i++) {
                for (int j = 0; j < n; j++) {
                    printf("%c", has_queen[i][j] ? 'Q' : '.');
                }
                printf("\n");
            }
        }
    } else {
        if (queens_left > 0 and no queen can attack cell (x,y)) {
            // try putting a queen on this cell
            has_queen[x][y] = true;
            queens_left--;

            generate(x, y+1);

            // undo the move
            has_queen[x][y] = false;
            queens_left++;
        }

        // try leaving this cell empty
        generate(x, y+1);
    }
}
```

Example problem: UVa 729 The Hamming Distance Problem

The Hamming distance between two strings of bits (binary integers) is the number of corresponding bit positions that differ. This can be found by using XOR on corresponding bits or equivalently, by adding corresponding bits (base 2) without a carry. For example, in the two bit strings that follow:

A	0	1	0	0	1	0	1	0	0	0
B	1	1	0	1	0	1	0	1	0	0
A XOR B =	1	0	0	1	1	1	1	1	0	0

The Hamming distance (H) between these 10-bit strings is 6, the number of 1's in the XOR string.

Input

Input consists of several datasets. The first line of the input contains the number of datasets, and it's followed by a blank line. Each dataset contains N , the length of the bit strings and H , the Hamming distance, on the same line. There is a blank line between test cases.

Example problem: UVa 729 The Hamming Distance Problem

Output

For each dataset print a list of all possible bit strings of length N that are Hamming distance H from the bit string containing all 0's (origin). That is, all bit strings of length N with exactly H 1's printed in ascending lexicographical order.

The number of such bit strings is equal to the combinatorial symbol $C(N, H)$. This is the number of possible combinations of $N - H$ zeros and H ones. It is equal to

$$\frac{N!}{(N - H)!H!}$$

This number can be very large. The program should work for $1 \leq H \leq N \leq 16$.
Print a blank line between datasets.

Sample Input

1

4 2

Sample Output

0011
0101
0110
1001
1010
1100

```

void generate(int at) {

    if (bits_set > k) {
        return;
    }

    if (at == n) {

        if (bits_set == k) {
            for (int i = 0; i < n; i++) {
                if (bit_set[i]) {
                    printf("1");
                } else {
                    printf("0");
                }
            }

            printf("\n");
        }

    } else {

        bit_set[at] = false;
        generate(at + 1);

        bit_set[at] = true;
        bits_set++;
        generate(at + 1);
        bits_set--;
    }

}

```

```

int main()
{
    int ts;
    scanf("%d\n", &ts);

    for (int t = 0; t < ts; t++) {

        if (t != 0) {
            printf("\n");
        }

        scanf("%d %d\n", &n, &k);

        bits_set = 0;
        memset(bit_set, 0, sizeof(bit_set));

        generate(0);

    }

    return 0;
}

```

Divide and conquer

- Given an instance of the problem, the basic idea is to
 1. split the problem into one or more smaller subproblems
 2. solve each of these subproblems recursively
 3. combine the solutions to the subproblems into a solution of the given problem
- Some standard divide and conquer algorithms:
 - Quick sort
 - Merge sort
 - Karatsuba algorithm
 - Strassen algorithm
 - Many algorithms from computational geometry
 - Convex hull
 - Closest pair of points

Divide and conquer: Time complexity

```
void solve(int n) {  
    if (n == 0)  
        return;  
  
    solve(n/2);  
    solve(n/2);  
  
    for (int i = 0; i < n; i++) {  
        // some constant time operations  
    }  
}
```

- ▶ What is the time complexity of this divide and conquer algorithm?
- ▶ Usually helps to model the time complexity as a recurrence relation:
 - $T(n) = 2T(n/2) + n$

Divide and conquer: Time complexity

- But how do we solve such recurrences?
- Usually simplest to use the Master theorem when applicable
 - It gives a solution to a recurrence of the form $T(n) = aT(n/b) + f(n)$ in asymptotic terms
 - All of the divide and conquer algorithms mentioned so far have a recurrence of this form
- The Master theorem tells us that $T(n) = 2T(n/2) + n$ has asymptotic time complexity $O(n \log n)$
- You don't need to know the Master theorem for this course, but still recommended as it's very useful

Decrease and conquer

- Sometimes we're not actually dividing the problem into many subproblems, but only into one smaller subproblem
- Usually called decrease and conquer
- The most common example of this is binary search

Binary search

- We have a sorted array of elements, and we want to check if it contains a particular element x
- Algorithm:
 1. Base case: the array is empty, return false
 2. Compare x to the element in the middle of the array
 3. If it's equal, then we found x and we return true
 4. If it's less, then x must be in the left half of the array
 - 4.1 Binary search the element (recursively) in the left half
 5. If it's greater, then x must be in the right half of the array
 - 5.1 Binary search the element (recursively) in the right half

Binary search

```
bool binary_search(const vector<int> &arr, int lo, int hi, int x) {  
    if (lo > hi) {  
        return false;  
    }  
  
    int m = (lo + hi) / 2;  
    if (arr[m] == x) {  
        return true;  
    } else if (x < arr[m]) {  
        return binary_search(arr, lo, m - 1, x);  
    } else if (x > arr[m]) {  
        return binary_search(arr, m + 1, hi, x);  
    }  
}  
  
binary_search(arr, 0, arr.size() - 1, x);
```

- ▶ $T(n) = T(n/2) + 1$
- ▶ $O(\log n)$

Binary search - iterative

```
bool binary_search(const vector<int> &arr, int x) {  
    int lo = 0,  
        hi = arr.size() - 1;  
  
    while (lo <= hi) {  
        int m = (lo + hi) / 2;  
        if (arr[m] == x) {  
            return true;  
        } else if (x < arr[m]) {  
            hi = m - 1;  
        } else if (x > arr[m]) {  
            lo = m + 1;  
        }  
    }  
  
    return false;  
}
```

Binary search over integers

- This might be the most well known application of binary search, but it's far from being the only application
- More generally, we have a predicate

$p : \{0, \dots, n-1\} \rightarrow \{T, F\}$ which has the property that if $p(i) = T$, then $p(j) = T$ for all $j > i$

- Our goal is to find the smallest index j such that $p(j) = T$ as quickly as possible

i	0	1	$\dots$	$j-1$	j	$j+1$	$\dots$	$n-2$	$n-1$
$p(i)$	F	F	$\dots$	F	T	T	$\dots$	T	T

- We can do this in $O(\log(n) \times f)$ time, where f is the cost of evaluating the predicate p , in the same way as when we were binary searching an array

Binary search over integers

```
int lo = 0,
    hi = n - 1;

while (lo < hi) {
    int m = (lo + hi) / 2;

    if (p(m)) {
        hi = m;
    } else {
        lo = m + 1;
    }
}

if (lo == hi && p(lo)) {
    printf("lowest index is %d\n", lo);
} else {
    printf("no such index\n");
}
```

Binary search over integers

- Find the index of x in the sorted array arr

```
bool p(int i) {  
    return arr[i] >= x;  
}
```

- Later we'll see how to use this in other ways

Binary search over reals

- An even more general version of binary search is over the real numbers
- We have a predicate $p : [\text{lo}, \text{hi}] \rightarrow \{T, F\}$ which has the property that if $p(i) = T$, then $p(j) = T$ for all $j > i$
- Our goal is to find the smallest real number j such that $p(j) = T$ as quickly as possible
- Since we're working with real numbers (hypothetically), our $[\text{lo}, \text{hi}]$ can be halved infinitely many times without ever becoming a single real number
- Instead it will suffice to find a real number j' that is very close to the correct answer j , say not further than $\text{EPS} = 2^{-30}$ away
- We can do this in $O(\log(\frac{\text{hi} - \text{lo}}{\text{EPS}}))$

time in a similar way as when we were binary searching an array

Binary search over reals

```
double EPS = 1e-10,  
       lo = -1000.0,  
       hi = 1000.0;  
  
while (hi - lo > EPS) {  
    double mid = (lo + hi) / 2.0;  
  
    if (p(mid)) {  
        hi = mid;  
    } else {  
        lo = mid;  
    }  
}  
  
printf("%.10lf\n", lo);
```

Binary search over reals

- ▶ This has many cool numerical applications
- ▶ Find the square root of x

```
bool p(double j) {  
    return j*j >= x;  
}
```

- ▶ Find the root of an increasing function $f(x)$

```
bool p(double x) {  
    return f(x) >= 0.0;  
}
```

- ▶ This is also referred to as the Bisection method

Example problem

- Problem C from NWERC 2006: Pie

Input

One line with a positive integer: the number of test cases. Then for each test case:

- One line with two integers N and F with $1 \leq N, F \leq 10\,000$: the number of pies and the number of friends.
- One line with N integers r_i with $1 \leq r_i \leq 10\,000$: the radii of the pies.

Output

For each test case, output one line with the largest possible volume V such that me and my friends can all get a pie piece of size V . The answer should be given as a floating point number with an absolute error of at most 10^{-3} .

Sample input	Sample output
3	25.1327
3 3	3.1416
4 3 3	50.2655
1 24	
5	
10 5	
1 4 2 3 4 5 6 5 4 2	

Binary search the answer

- It may be hard to find the optimal solution directly, as we saw in the example problem
- On the other hand, it may be easy to check if some x is a solution or not
- A method of using binary search to find the minimum or maximum solution to a problem
- Only applicable when the problem has the binary search property: if i is a solution, then so are all $j > i$
- $p(i)$ checks whether i is a solution, then we simply apply binary search on p to get the minimum or maximum solution

Other types of divide and conquer

- Binary search is very useful, can be used to construct simple and efficient solutions to problems
- But binary search is only one example of divide and conquer
- Let's explore two more examples

Binary exponentiation

- We want to calculate x^n , where x, n are integers
- Assume we don't have the built in pow method
- Naive method:

```
int pow(int x, int n) {  
    int res = 1;  
    for (int i = 0; i < n; i++) {  
        res = res * x;  
    }  
  
    return res;  
}
```

- This is $O(n)$, but what if we want to support large n efficiently?

Binary exponentiation

- Let's use divide and conquer

- Notice the three identities:

-

$$x^0 = 1$$

$$x^n = x \times x^{n-1}$$

$$x^n = x^{(n/2)} \times x^{(n/2)}$$

- Or in terms of our function:

$$\text{pow}(x,0)=1$$

$$\text{pow}(x,n) = x \times \text{pow}(x,n - 1)$$

$$\text{pow}(x,n)=\text{pow}(x,n/2)\times\text{pow}(x,n/2)$$

- $\text{pow}(x, n/2)$ is used twice, but we only need to compute it once:

$$\text{pow}(x,n)=\text{pow}(x,n/2)^2$$

Binary exponentiation

- Let's try using these identities to compute the answer recursively

```
int pow(int x, int n) {  
    if (n == 0) return 1;  
    return x * pow(x, n - 1);  
}
```

- How efficient is this?

Binary exponentiation

- Let's try using these identities to compute the answer recursively

```
int pow(int x, int n) {  
    if (n == 0) return 1;  
    return x * pow(x, n - 1);  
}
```

- How efficient is this?
 - $T(n) = 1 + T(n-1)$
 - $O(n)$
 - Still just as slow...

Binary exponentiation

- What about the third identity?
 - $n/2$ is not an integer when n is odd, so let's only use it
- when n is even

```
int pow(int x, int n) {  
    if (n == 0) return 1;  
    if (n % 2 != 0) return x * pow(x, n - 1);  
    int st = pow(x, n/2);  
    return st * st;  
}
```

- How efficient is this?

Binary exponentiation

- What about the third identity?
 - $n/2$ is not an integer when n is odd, so let's only use it
- when n is even

```
int pow(int x, int n) {  
    if (n == 0) return 1;  
    if (n % 2 != 0) return x * pow(x, n - 1);  
    int st = pow(x, n/2);  
    return st * st;  
}
```

- How efficient is this?
 - $T(n) = 1 + T(n-1)$ if n is odd
 - $T(n) = 1 + T(n/2)$ if n is even
 - Since $n-1$ is even when n is odd
 - $T(n) = 1 + 1 + T((n-1)/2)$ if n is odd
 - $O(\log n)$
 - Fast!

Binary exponentiation

- Notice that x doesn't have to be an integer, and $*$ doesn't have to be integer multiplication...
- It also works for:
 - Computing x^n , where x is a floating point number and $*$ is floating point number multiplication
 - Computing A^n , where A is a matrix and $*$ is matrix multiplication
 - Computing $x^n \pmod m$, where x is a matrix and $*$ is integer multiplication modulo m
 - Computing $x * x * \dots * x$, where x is any element and $*$ is any associative operator
- All of these can be done in $O(\log(n) \times f)$, where f is the cost of doing one application of the $*$ operator

Fibonacci words

- Recall that the Fibonacci sequence can be defined as follows:
 $\text{fib_1} = 1$
 $\text{fib_2} = 1$
 $\text{fib_n} = \text{fib_}(n-2) + \text{fib_}(n-1)$
- We get the sequence 1,1,2,3,5,8,13,21,...
- There are many generalizations of the Fibonacci sequence
- One of them is to start with other numbers, like:
 $f_1 = 5$
 $f_2 = 4$
 $f_n = f_ (n-2) + f_ (n-1)$
- We get the sequence 5,4,9,13,22,35,57,...
- What if we start with something other than numbers?

Fibonacci words

- Let's try starting with a pair of strings, and let $+$ denote string concatenation:

$$g_1 = A$$

$$g_2 = B$$

$$g_n = g_{(n-2)} + g_{(n-1)}$$

- Now we get the sequence of strings:

A

B

AB

BAB

ABBAB

BABABBAB

ABBABBABABBAB

BABABBABABBABBABABBAB

...

Fibonacci words

- How long is g_n ?
 - $\text{len}(g_1) = 1$
 - $\text{len}(g_2) = 1$
 - $\text{len}(g_n) = \text{len}(g_{n-2}) + \text{len}(g_{n-1})$
- Looks familiar?
- $\text{len}(g_n) = \text{fib}_n$
- So the strings become very large very quickly
 - $\text{len}(g_{10}) = 55$
 - $\text{len}(g_{100}) = 354224848179261915075$
 - $\text{len}(g_{1000}) =$
434665576869374564356885276750406258025646605173717
804024817290895365554179490518904038798400792551692
959225930803226347752096896232398733224711616429964
409065331879382989696499285160037044761377951668492
28875

Fibonacci words

- Task: Compute the i -th character in g_n

Fibonacci words

- Task: Compute the i -th character in g_n
- Simple to do in $O(\text{len}(n))$, but that is extremely slow for large n
- Can be done in $O(n)$ using divide and conquer